

Smart Room IoT Monitoring

Dashboard Monitoring Ruangan Real-time berbasis IoT + Integrasi Machine Learning

Panduan & Script Presentasi - Internet of Things, Semester 5

Konten: penjelasan tiap bagian yang dikerjakan, keputusan teknis & alasan, plus narasi siap diucapkan (estimasi waktu per bagian).

Bagian 0 - Ringkasan Proyek (Slide 1)

Sistem yang saya bangun adalah smart room monitoring berbasis IoT. ESP32 membaca sensor DHT11 (suhu & kelembapan) dan MQ-135 (kualitas udara / asap), lalu mengendalikan aktuator (LED merah/hijau, buzzer, relay fan). Data dikirim lewat WiFi ke server web (Next.js) menggunakan HTTP POST JSON tiap 10 detik, disimpan di database lokal SQLite, dan ditampilkan di dashboard web secara real-time.

Yang membuat sistem ini menarik: logika alert (LED, buzzer, fan) tetap berjalan LOKAL di ESP32. Kalau WiFi atau server mati, perlindungan ruangan tetap jalan. Server hanya menerima, menyimpan, menampilkan, dan mengirim perintah kontrol.

Narasi (kurang lebih 1 menit):

"Selamat pagi/siang. Proyek saya adalah sistem monitoring ruangan berbasis IoT. ESP32 membaca suhu, kelembapan, dan kualitas udara, lalu jika terdeteksi kondisi berbahaya, sistem langsung menyalakan buzzer, LED merah, dan kipas -- dan yang penting, logika ini tetap bekerja meskipun WiFi atau server mati, karena semua keputusan juga diambil lokal di perangkat. Dashboard web-nya saya buat berbasis Next.js, yang menerima data tiap 10 detik, menyimpannya, dan menampilkannya secara real-time. Nanti saya jelaskan detail tiap bagian dan alasan di balik setiap keputusan teknis."

Bagian 1 - Arsitektur Sistem (Slide 2)

Apa yang dikerjakan

Saya merancang arsitektur dengan alur data satu arah dan dua loop kontrol:

- ESP32 -> Next.js API -> SQLite -> Dashboard: sensor dibaca 2 detik sekali, dikirim ke server tiap 10 detik, divisualisasikan ke dashboard.
- Loop kontrol aktuator (dashboard -> ESP32): dashboard menulis perintah ke tabel `device_commands`, ESP32 mengecek tiap 15 detik, mengeksekusi, lalu melaporkan ACK.
- Loop kontrol auto (threshold/AI): server menghitung status dari threshold yang bisa dikalibrasi, lalu mengantrekan perintah otomatis ke ESP32. Firmware juga menyinkronkan threshold dari server tiap 30 detik.

Keputusan teknis & alasan

- Beberapa logika tetap di ESP32 (offline-proof) -- prinsip utama dari PRD: perlindungan tidak boleh bergantung pada koneksi internet.
- Frontend dan backend dalam satu project Next.js (App Router) -- cukup satu project, satu proses deploy, dan tipe data bisa dipakai bersama.
- Real-time dengan polling sederhana, bukan WebSocket -- kebutuhan hanya menampilkan data yang datang tiap 10 detik. Polling 5 detik cukup memenuhi kriteria 'data muncul di dashboard kurang dari 5 detik', tanpa perlu koneksi persisten atau server push.
- Dua jalur kontrol (manual vs otomatis) dipisah dengan jelas dan ada mekanisme prioritas (grace period manual 15 detik di firmware), supaya pengguna tidak berkelahi dengan logika auto.

Narasi (kurang lebih 1 menit 30 detik):

"Arsitekturnya saya sederhanakan menjadi alur data satu arah. ESP32 mengirim data ke API Next.js, lalu disimpan ke SQLite dan ditampilkan di dashboard. Untuk kontrol, ada dua loop: yang pertama dari dashboard ke ESP32 lewat perintah berantre, dan yang kedua dari server yang otomatis mengatur aktuator berdasarkan threshold. Saya sengaja menyimpan keputusan alert di ESP32 juga, supaya ketika internet mati sistem tetap aman. Untuk real-time, saya memilih polling ketimbang WebSocket karena interval datanya 10 detik, sehingga polling 5 detik sudah terasa real-time tanpa menjaga koneksi permanen."

Bagian 2 - Teknologi yang Dipakai & Alasannya (Slide 3)

Layer	Teknologi	Alasan
Framework	Next.js (App Router + TypeScript)	API & dashboard 1 project, type-safe
Styling	Tailwind CSS v4	Cepat, konsisten, mudah dibuat responsif
Chart	Recharts	Ringan, native React, cocok untuk line chart tren
Database	SQLite (better-sqlite3)	Lokal, tanpa setup cloud, cukup untuk demo
Validasi	Zod	Payload dari ESP32 wajib divalidasi sebelum masuk DB
Firmware	Arduino (WiFi.h + HTTPClient + ArduinoJson)	File JSON kecil, hemat RAM di ESP32
ML (eksperimen)	Python: pandas, scikit-learn, statsmodels	Analisis data & eksperimen model
ML (produksi)	TypeScript in-app (lib/ml.ts)	Berjalan langsung di server tanpa proses tambahan

Keputusan penting yang bisa ditanyakan

Mengapa migrasi dari Supabase ke SQLite?

Di versi awal (PRD) saya menggunakan Supabase/Postgres. Saat pengembangan lanjutan, saya pertimbangkan kebutuhan: data hanya dari satu device, kualitas datanya real-time cukup di-poll, dan dashboard dipakai lokal. SQLite memberi keuntungan: tanpa registrasi cloud, tanpa latensi jaringan, data dalam satu file (data/smartroom.db), setup instan lewat npm install + npm run dev. Skema tabel pun dibuat otomatis saat server pertama jalan. Trade-off-nya jelas saya catat: SQLite tidak cocok untuk produksi serverless multi-user; untuk itu tetap dibutuhkan Postgres/Supabase. Dalam demo ini, lokal SQLite adalah keputusan yang paling sederhana dan andal.

Mengapa polling, bukan WebSocket atau Supabase Realtime?

- Data masuk tiap 10 detik -- kebutuhan menampilkan ulang sangat rendah.
- Polling HTTP stateless: mudah diuji, tidak ada koneksi yang harus dijaga, tidak ada infrastruktur server tambahan.
- Kriteria acceptance (data muncul < 5 detik) terpenuhi dengan polling 5 detik.
- Supabase Realtime hanya berguna bila data dikirim oleh banyak sumber bersamaan -- di sini tidak.

Mengapa Zod?

Perangkat IoT adalah sumber data yang luar biasa -- kadang mengirim NaN, string, atau nilai di luar rentang. Zod membuat kontrak yang ketat: temperature & humidity wajib number (bukan NaN), gas_value integer 0-4095, boolean opsional. Payload tak valid ditolak dengan status 400 dan pesan yang jelas.

Narasi (kurang lebih 1 menit 30 detik):

"Untuk teknologi, inti saya pakai Next.js agar API dan dashboard jadi satu project. Database-nya saya buat lokal dengan SQLite; awalnya proyek ini memakai Supabase, tapi karena data datang hanya dari satu perangkat dan kebutuhan real-time-nya sederhana, SQLite lebih cepat dan tanpa biaya cloud. Data tersimpan dalam satu file dan tabelnya otomatis dibuat saat server pertama berjalan. Validasi payload ESP32 saya kerjakan dengan Zod supaya data rusak seperti NaN tidak pernah masuk database, dan untuk real-time saya pakai polling lima detik karena interval pengiriman datanya sepuluh detik."

Bagian 3 - Firmware ESP32: Inti (Slide 4-6)

Rangkaian & pin (sumber kebenaran)

Komponen	Pin ESP32	Catatan
DHT11 (suhu & kelembapan)	GPIO 4 (DATA)	3V3 - GND
MQ-135 (kualitas udara)	GPIO 34 (AO, ADC1)	VCC 5V/VIN, ADC input-only
LED merah (indikator bahaya)	GPIO 25	via resistor 220 ohm
LED hijau (indikator normal)	GPIO 26	via resistor 220 ohm
Buzzer	GPIO 27	
Relay / fan	GPIO 17	Active-LOW, lihat catatan

Catatan relay: modul relay ini Active-LOW dengan pull-up internal ke 5V. GPIO LOW = relay ON (fan menyala), GPIO di-set INPUT (ngambang) = relay OFF. Yang penting: JANGAN memakai digitalWrite HIGH untuk mematikan, karena GPIO ESP32 cuma 3.3V sedangkan modul butuh 5V penuh untuk dianggap OFF.

Pembacaan sensor non-blocking (millis)

Loop utama tidak memakai delay() agar tetap responsif. Ada 4 timer independen berbasis millis: baca sensor tiap 2 detik, kirim ke server tiap 10 detik, polling perintah dashboard + status AI tiap 15 detik, dan sinkronisasi threshold tiap 30 detik.

Logika alert lokal (offline-proof)

ESP32 menyimpan threshold sendiri dan mengevaluasi sensor tiap 2 detik. Jika suhu, kelembapan, atau gas melewati threshold, maka LED merah + buzzer + fan langsung aktif -- ini berjalan penuh tanpa internet. Kirim data ke server hanyalah 'bonus' untuk pencatatan; kalau gagal, ESP32 cukup mencatat kegagalan dan mencoba lagi.

Kirim data dengan autentikasi

Setiap POST /api/telemetry menyertakan header x-api-key yang harus sama persis dengan IOT_API_KEY di .env.local -- jika salah, server menolak dengan 401. Body dikirim sebagai JSON berisi device_id, temperature, humidity, gas_value, serta state aktuator (fan_on, buzzer_on, led_red_on, led_green_on).

Narasi (kurang lebih 2 menit):

"Sekarang ke firmware. Perangkat kerasnya sederhana: satu DHT11 untuk suhu dan kelembapan, satu MQ-135 untuk kualitas udara, lalu dua LED, buzzer, dan relay kipas. Satu hal penting soal relay: modul yang saya pakai adalah active-low, jadi menyalakan kipas berarti menurunkan pin, dan mematikannya berarti pin dibuat mengambang -- karena GPIO 3.3 volt tidak cukup kuat untuk modul 5 volt. Loop utama saya tulis non-blocking memakai millis, jadi membaca sensor, mengirim data, dan mengecek perintah berjalan bergantian tanpa saling menahan. Yang paling penting: pengecekan bahaya dilakukan lokal di perangkat, jadi kalau WiFi mati atau server mati, buzzer dan kipas tetap bekerja. Setiap kirim data juga saya autentikasi dengan API key supaya hanya perangkat saya yang bisa mengirim."

Bagian 4 - Firmware: Fitur Unggulan yang Baru Dikembangkan (Slide 5, 8, 12)

1) Auto-discovery server

Sebelumnya setiap ganti jaringan harus mengganti IP laptop di kode firmware. Sekarang firmware otomatis memindai jaringan (fungsi probeServer / discoverServer) saat boot untuk menemukan server dashboard, dan hanya menggunakan SERVER_HOST sebagai fallback. Karena itu cukup flash sekali dan firmware langsung menemukan server di jaringan yang sama.

2) Sinkronisasi threshold dari dashboard

Threshold sekarang milik server (single source of truth) dan disinkronkan ke ESP32 tiap 30 detik lewat GET /api/settings. Artinya kalibrasi yang dilakukan dari halaman dashboard langsung dipakai juga oleh logika lokal ESP32 -- tanpa perlu flash ulang.

3) Auto-control dengan mode yang jelas

- manualMode=false secara default: logika auto lokal ON (mengikuti threshold).
- cloudAutoControl: ikut toggle di dashboard (autoControl di /api/settings). Jika mati, server berhenti mengantrekan perintah otomatis.
- Manual override: setiap perintah manual/dashboard menetapkan manualOverrideUntil = 15 detik ke depan, sehingga output yang kita pilih tidak langsung ditimpa logika auto.
- Mode autocloud: fan mengikuti klasifikasi AI (NORMAL/WASPADA/BAHAYA) lewat polling /api/ml tiap 15 detik.

4) Perintah Serial untuk debugging

Serial Monitor menerima perintah: hijau, merah, fan, buzzer, auto, autocloud, status (menampilkan threshold & server aktif), dan cari (memindai ulang server).

Narasi (kurang lebih 2 menit):

"Ada empat kemampuan firmware yang saya tambahkan. Pertama, auto-discovery: saya tidak perlu lagi mengubah alamat IP di kode setiap pindah jaringan, karena ESP32 memindai jaringan dan menemukan server secara otomatis. Kedua, sinkronisasi threshold: saya menjadikan server sebagai sumber kebenaran tunggal, dan ESP32 menarik nilai threshold setiap tiga puluh detik -- jadi kalau saya mengubah batas gas dari dashboard, logika lokal di perangkat ikut berubah tanpa menulis ulang firmware. Ketiga, auto-control dengan prioritas: logika otomatis menyala secara default, tetapi setiap perintah manual diberi masa tenggang lima belas detik supaya tidak langsung tertimpa. Dan keempat, mode autocloud di mana kipas mengikuti hasil AI. Semua ini bisa saya uji lewat Serial Monitor dengan perintah seperti status dan cari."

Bagian 5 - Backend API (Slide 2, 8, 12)

Apa yang dikerjakan

Route	Fungsi
POST /api/telemetry	Ingest dari ESP32: validasi Zod + API key, hitung status, simpan, buat alert
GET /api/telemetry?limit	Riwayat data (default 100, max 500, filter from/to)
GET /api/telemetry/latest	1 data terbaru untuk kartu real-time
GET /api/alerts?limit	Log alert terbaru
GET /api/stats?hours	Agregasi avg/min/max + jumlah alert
GET/POST/PATCH /api/actuator	Kelola perintah aktuator (web control + polling ESP32 + ACK)
GET/POST /api/settings	Baca/simpan kalibrasi threshold + auto-control
GET /api/ml	Klasifikasi & prediksi AI (in-app)

Logika utama di POST /api/telemetry

- Keamanan: header x-api-key dibandingkan dengan env IOT_API_KEY -> salah = 401.
- Validasi: skema Zod -> salah = 400 dengan pesan field apa yang gagal.
- Status dihitung dari threshold yang TERSIMPAN di database (bukan hardcoded): gas/temp/hum melampaui ambang -> status ALERT. Satu fungsi computeStatus di lib/thresholds.ts menjadi satu-satunya tempat menentukan NORMAL/ALERT.
- Saat ALERT: untuk setiap pelanggaran (GAS/TEMP/HUM) dibuat satu baris di tabel alerts berisi nilai aktual vs threshold yang dipakai -- transparan dan mudah dijelaskan.
- Auto-control terpusat (queueAutoControl): jika autoControl aktif, server mengantrekan perintah (fan_on, buzzer_on, led_red_on, led_green_off, dst) ke device_commands. Penting: hanya saat state BERUBAH dari laporan ESP32, dan dicegat duplikat jika masih ada perintah pending sejenis -- mencegah spam tiap 10 detik.

Web control aktuator (alur lengkap)

Klik toggle di dashboard -> POST /api/actuator menyimpan perintah ke device_commands (executed=0) -> ESP32 polling GET /actuator tiap 15 detik -> mengeksekusi pin -> melaporkan ACK lewat PATCH /actuator -> dashboard menampilkan status terbaru.

Narasi (kurang lebih 2 menit):

"Backendnya berupa API Next.js. Titik masuk utama adalah POST telemetry. Saya menjaga keamanan dengan membandingkan API key, lalu memvalidasi seluruh payload dengan Zod sebelum menyimpan. Status normal atau alert dihitung dari threshold yang tersimpan di database, bukan nilai hardcoded, sehingga bisa dikalibrasi. Kalau statusnya alert, untuk setiap pelanggaran saya catat satu baris alert beserta nilai dan ambangnya, dan yang menarik: server bisa mengantrekan perintah otomatis ke ESP32 -- tapi hanya ketika keadaan benar-benar berubah sukar untuk mencegah perintah berulang setiap sepuluh detik. Untuk mengontrol aktuator dari dashboard, saya pakai model perintah berantre: dashboard menulis perintah, ESP32 mengambilnya tiap lima belas detik, mengeksekusi, lalu melaporkan hasilnya."

Bagian 6 - Database & Real-time (Slide 2, 6, 14)

Apa yang dikerjakan

Skema database dibuat otomatis di lib/db.ts memakai better-sqlite3 dengan mode WAL. Baris data disimpan di file data/smartroom.db. Tabel yang dibuat:

- telemetry -- satu baris per pembacaan 10 detik (nilai sensor + status + state aktuator).
- alerts -- catatan pelanggaran dengan alert_type, nilai vs threshold.
- device_commands -- antrean perintah aktuator (manual & auto) untuk di-poll ESP32.
- ml_classifications & ml_predictions -- hasil klasifikasi/prediksi AI.
- settings -- key-value untuk kalibrasi threshold & auto-control (di-seed dengan default: gas 500, temp 32.0, hum 75.0, auto-control ON).

Indeks dibuat pada kolom yang sering dipakai: created_at, device_id, dan executed untuk antrean perintah. Timestamp memakai datetime lokal server agar mudah dibaca.

Kenapa real-time pakai polling, bukan push

- Interval data 10 detik -> polling 5 detik sudah real-time secara visual.
- Tanpa koneksi persisten: tidak ada WebSocket yang harus dijaga, tidak ada server realtime tambahan, lebih mudah di-debug.
- Fallback otomatis: kalau polling gagal (server restart), browser cukup mencoba lagi di interval berikutnya.
- Offline detection: jika data terakhir lebih dari 30 detik, dashboard menampilkan badge OFFLINE -- ini logika sederhana dari polling yang ada.

Narasi (kurang lebih 1 menit):

"Data saya simpan di SQLite lokal, dan tabelnya saya buat otomatis. Ada tabel untuk telemetry, alert, antrean perintah aktuator, hasil AI, dan pengaturan threshold yang dirancang sedemikian rupa -- pakai indeks pada kolom waktu dan id agar query cepat. Untuk real-time saya pilih polling daripada push karena interval datanya sepuluh detik; polling lima detik membuat dashboard terasa hidup tanpa harus menjaga koneksi permanen. Deteksi perangkat offline juga saya bangun dari polling ini -- kalau tidak ada data selama lebih dari tiga puluh detik, statusnya berubah menjadi offline."

Bagian 7 - EDA & Penentuan Threshold dari Data Riil (Slide 13)

Apa yang dikerjakan

Daripada menebak nilai threshold, saya melakukan Exploratory Data Analysis terhadap 855 baris data riil yang sudah terkumpul: statistik deskriptif, korelasi, dan plot distribusi (histogram, boxplot, trend). Tujuannya untuk menentukan batas NORMAL / WASPADA / BAHAYA dari kondisi ruangan yang sebenarnya.

Hasil statistik & korelasi

Sensor	Statistik (data riil)
Suhu	mean 31.73 C, std 1.02, range 28.4 - 33.8 C
Kelembapan	mean 67.46%, std 6.14, range 56 - 76%
Gas	mean 106 ADC, std 172, max 3900 (spike)

- Korelasi suhu vs kelembapan = -0.94: kuat negatif (makin panas makin kering) -- fisiknya masuk akal.
- Korelasi gas vs suhu/hum sangat lemah: gas bersifat independen.

Threshold final & alasannya

Parameter	Nilai	Dasar keputusan
TEMP_WASPADA	33.0 C	Data normal 31-33 C, p95 = 32.8 C (bukan 30 yang terlalu agresif)
TEMP_BAHAYA	36.0 C	Referensi hangat ekstrem
HUM_LOW	30.0%	Referensi kenyamanan (data tidak pernah < 59%)
HUM_HIGH	75.0%	Mendekati max data 76%
GAS_WASPADA	1500 ADC	Referensi MQ-135
GAS_BAHAYA	2500 ADC	Referensi MQ-135

Catatan: ada dua lapis threshold. (1) Alert sistem (computeStatus) memakai threshold kalibrasi di dashboard -- default gas 500, temp 32, hum 75 -- untuk keputusan cepat yang konservatif. (2) Klasifikasi ML memakai level WASPADA/BAHAYA hasil EDA di atas untuk kategorisasi NORMAL/WASPADA/BAHAYA. Keduanya didefinisikan dari data, bukan asal tebak.

Narasi (kurang lebih 1 menit 30 detik):

"Sebelum menentukan ambang bahaya, saya menganalisis dulu data riil yang sudah terkumpul -- delapan ratus lima puluh lima baris. Dari situ saya tahu suhu ruangan normalnya sekitar tiga puluh satu sampai tiga puluh tiga derajat, jadi saya menjadikan tiga puluh tiga derajat sebagai zona waspada, bukan nilai yang terlalu agresif. Korelasi suhu dan kelembapan yang sangat negatif juga membuktikan data itu masuk akal. Untuk gas, karena data bersih, ambangnya saya ambil dari referensi sensor MQ-135. Jadi keputusan threshold di sini berbasis data dan referensi, bukan tebak."

Bagian 8 - Machine Learning (Slide 7-11)

Jalur eksperimen: Python (analisis & validasi)

- Preprocessing (preprocessing.py): deteksi spike memakai metode IQR lalu penggantian forward-fill (18 spike terdeteksi), dan interpolasi linear untuk data hilang pada gap 10-300 detik (27 baris terisi). Gap yang lebih panjang (perangkat mati) dibiarkan.
- Feature engineering: untuk tiap sensor dibuat fitur mean rolling 10 baris, std rolling, delta antar baris, plus jam dalam sehari -> 13 kolom fitur. Alasannya: kondisi ruangan tidak tergantung nilai sesaat saja, tapi juga tren dan kecepatan perubahannya.
- Decision Tree (scikit-learn): max_depth 6, random_state 42; dilabeli rule-based hasil EDA; retrain tiap 60 detik dan disimpan ke dt_classifier.joblib.
- ARIMA (statsmodels): order (1,1,1), resample per jam, forecast 3 jam ke depan.

Jalur produksi: ML in-app di server (lib/ml.ts)

Agar demo berdiri sendiri tanpa proses Python terpisah, saya implementasikan kembali logika cerdas langsung di TypeScript server:

- classifyReading: skor berbasis threshold dengan multiplier bertingkat (mis. gas > threshold -> skor 1, > 1.15x -> 2, > 1.3x -> 3). Total >= 2 = WASPADA, >= 4 = BAHAYA, selain itu NORMAL; confidence dihitung dari skor. Alasan memakai rule-based: mudah dijelaskan, deterministik, dan langsung bisa memantik aktuator.
- PredictNextValues / linearRegressionPrediction: regresi linier dari 30 bacaan terakhir untuk memperkirakan nilai 5 menit ke depan, lalu diklasifikasikan lagi. Alasan memakai regresi linier: ringan, tanpa library, cukup untuk tren jangka pendek.

Kenapa dua jalur?

Python dipakai untuk eksplorasi, visualisasi, dan bukti konsep (bisa dibuat notebook atau plot). TypeScript in-app dipakai di produksi demo supaya server Next.js itu sendiri yang menghitung tanpa perlu menjalankan proses Python -- satu unit yang berjalan hanya dengan npm run dev.

Narasi (kurang lebih 2 menit):

"Bagian AI saya kerjakan dalam dua jalur. Jalur eksplorasi memakai Python untuk membersihkan data -- misalnya deteksi spike dengan metode IQR supaya lonjakan gas palsu tidak merusak model -- lalu membuat fitur turunan seperti rata-rata berjalan dan kecepatan perubahan. Model klasifikasinya Decision Tree dan prediksi trennya ARIMA. Kemudian, supaya demo tidak bergantung pada proses Python yang harus berjalan terpisah, saya tulis ulang logika yang sama di dalam server Next.js: klasifikasi berbasis skor threshold dan prediksi tren dengan regresi linier sederhana. Dengan begitu, kapan pun dashboard dibuka hasil AI langsung tersedia. Alasan saya memilih pendekatan berbasis aturan untuk klasifikasi adalah karena hasilnya deterministik dan mudah dijelaskan, sementara untuk prediksi jangka pendek regresi linier sudah cukup dan sangat ringan."

Bagian 9 - Dashboard Web Real-time (Slide 14)

Apa yang dikerjakan

- Header dengan badge LIVE/OFFLINE (dari timestamp data terakhir) dan timestamp update.
- Kartu sensor (Suhu, Kelembapan, Gas) dengan warna status dan label level gas (Good / Moderate / Poor / Hazardous).
- StatusBanner: NORMAL hijau / ALERT merah + daftar alasan.
- ActuatorCard: toggle interaktif Fan, Buzzer, LED merah/hijau; state 'menunggu ESP32' selama belum ada konfirmasi.
- TelemetryChart (Recharts): line chart tren temp/hum/gas 24 jam, auto-update.
- StatsPanel: avg/min/max + jumlah alert.
- DataLog (riwayat pembacaan), AlertTable (log alert: jenis, pesan, nilai vs threshold).
- ML cards: klasifikasi AI + prediksi bacaan berikutnya.
- ThresholdCard: kalibrasi threshold + toggle auto-control (dijelaskan di Bagian 10).

Keputusan & alasan

- Polling terpusat di page dengan interval berbeda (latest 5 dtk, riwayat & statistik 10-15 dtk, AI 30 dtk) -- hemat request, tetap real-time.
- Semua kartu memakai pola visual konsisten (HUD) sehingga mudah dibedakan statusnya.
- Responsif penuh: grid auto-fit sehingga pada layar sempit kartu otomatis menjadi satu kolom, dan tabel bisa digeser horizontal.

Narasi (kurang lebih 1 menit 30 detik):

"Dashboard-nya berupa satu halaman yang hidup. Ada kartu untuk tiap sensor dengan warna yang langsung menunjukkan kondisi, banner status normal atau alert, dan grafik tren dua puluh empat jam. Semua nilai ini diperbarui dengan polling -- data terbaru tiap lima detik, riwayat dan statistik tiap sepuluh sampai lima belas detik, dan hasil AI tiap tiga puluh detik. Saat alert terjadi, kartu dan banner berubah warna dan baris alert langsung muncul lengkap dengan nilai versus batasnya. Tampilannya saya buat responsif: di layar kecil semua kartu menumpuk jadi satu kolom tanpa ada yang terpotong."

Bagian 10 - Kalibrasi Threshold & Auto-Control dari Dashboard

Apa yang dikerjakan

Membuat ThresholdCard di dashboard yang mengirim POST /api/settings. Server menyimpannya ke tabel settings dan langsung menjadikannya acuan computeStatus. ESP32 menarik nilai yang sama tiap 30 detik, sehingga kalibrasi dari dashboard berlaku juga di logika lokal (tanpa flash ulang).

Alasan desain

- Single source of truth di server: semua pihak (status server, alert, ML in-app, dan ESP32) memakai nilai yang sama.
- Toggle auto-control: kalau dimatikan, server tidak lagi mengantrekan perintah otomatis -- pengguna memegang kendali penuh. Firmware tetap punya grace period 15 detik untuk perintah manual.
- UI jujur terhadap sistem: tombol menunjukkan status 'menunggu ESP32' sampai perintah di-ACK, sehingga tidak membohongi pengguna.

Narasi (kurang lebih 1 menit):

"Satu fitur yang saya rasa penting adalah kalibrasi threshold langsung dari dashboard. SP32 tidak perlu di-flash ulang ketika saya mengubah batas gas, suhu, atau kelembapan -- cukup ubah di kartu kalibrasi, server menyimpannya sebagai sumber kebenaran, dan dalam tiga puluh detik ESP32 ikut berubah. Saya juga menambahkan toggle auto-control; jika dimatikan, server berhenti mengeluarkan perintah otomatis sehingga pengguna bisa memegang kendali penuh. Dan seluruh status di dashboard itu jujur -- kalau lampu masih menunggu konfirmasi dari perangkat, ditampilkan sebagai sedang ditunggu."

Bagian 11 - Penyesuaian UI Agar Responsif & Konsisten

Apa yang dikerjakan

- Font: memakai font Geist bawaan Next.js, konsisten antar bagian.
- Pola HUD terpusat: `.hud-panel`, `.hud-title`, `.hud-chip`, `.hud-note` didefinisikan di `@layer components` supaya utility Tailwind masih bisa memodifikasinya (mis. border warna per status).
- Layout fleksibel: `grid auto-fit (repeat(auto-fit, minmax(...)))` -- jumlah kolom menyesuaikan ruang yang tersedia, bukan breakpoint kaku.
- Tipografi fluida: nilai sensor memakai `clamp()` sehingga ukuran teks mengikuti lebar layar dengan halus, tidak lompat-lompat.
- Tabel bisa digeser horizontal di layar kecil (`overflow-auto + min-w`) dengan scrollbar tipis.

Narasi (kurang lebih 40 detik):

"Terakhir, saya merapikan tampilan supaya konsisten dan fleksibel. Semua kartu memakai pola visual yang sama, warna status bisa mengubah tepi kartu. Grid-nya saya buat menyesuaikan diri -- bukan lagi tiga kolom kaku, tapi kolom terbentuk sesuai ruang yang ada, jadi di tablet muncul dua kolom dan di ponsel satu kolom dengan ukuran teks yang ikut menyesuaikan."

Bagian 12 - Kelebihan Sistem & Cara Menjalankan (Slide 16-17)

Kelebihan yang bisa ditekankan

Kelebihan	Penjelasan
Offline-proof	Alert lokal di ESP32 jalan walau WiFi/server mati
Real-time	Polling 5 detik; data muncul di dashboard cepat
AI-powered	Klasifikasi status + prediksi tren nilai sensor
Threshold data-driven	Batas ditentukan dari EDA data riil, bukan tebakan
Kalibrasi tanpa flash	Threshold & auto-control diubah dari dashboard, ESP32 ikut 30 detik
Auto-discovery	ESP32 menemukan server sendiri, tanpa ubah IP manual
Satu project	Dashboard + API + ML in-app cukup npm run dev

Cara menjalankan (demo)

- 1) Dashboard: `cd smart-room-dashboard && npm install && npm run dev`, buka `http://localhost:3000`.
- 2) Firmware: upload `smart_room_monitoring.ino` via Arduino IDE (board ESP32); atur `WIFI_SSID` & `WIFI_PASS`; `API_KEY` harus sama dengan `IOT_API_KEY` di `.env.local`. Server ditemukan otomatis (auto-discovery); `SERVER_HOST` hanya fallback.
- 3) Untuk kontrol dari laptop: pastikan HP/ESP32 dan laptop di jaringan yang sama.
- 4) Demo: ubah threshold di dashboard lalu amati ESP32 ikut; matikan WiFi utk perlihatkan offline-proof (LED/buzzer tetap jalan).

Narasi penutup (kurang lebih 1 menit):

"Jadi sistem ini punya beberapa keunggulan yang mau saya tekankan: alert tetap jalan saat internet mati, tampilan real-time, ada AI untuk klasifikasi dan prediksi, threshold yang berasal dari data riil, dan kalibrasi yang tidak butuh flash ulang. Supaya singkat, untuk demo saya cukup menjalankan `npm run dev` dan menyalakan ESP32 -- server langsung ditemukan dengan auto-discovery. Terima kasih, saya siap menerima pertanyaan."

Bagian 13 - Penutup & Referensi (Slide 18)

Ringkasan yang disampaikan

Dari sensor hingga dashboard, seluruh rantai dibuat lengkap: firmware ESP32 dengan logika lokal, backend Next.js dengan validasi & single source of truth, database SQLite, analisis data untuk threshold, integrasi AI, hingga dashboard real-time yang responsif dan memiliki kontrol aktuator.

Keterbatasan (jujur di depan penguji)

- Database SQLite berjalan di filesystem lokal -- untuk produksi cloud multi-user perlu Postgres/Supabase.
- Belum ada otentikasi multi-user; API key tunggal untuk perangkat.
- Prediksi hanya tren jangka pendek (5 menit in-app; 3 jam di Python); bukan model siklus penuh.
- Sensor DHT11 punya akurasi terbatas; untuk penelitian ketat bisa naik ke DHT22/AM2301B.

Referensi

PRD (prd.md), Panduan PPT (docs/panduan_ppt.txt), dokumentasi Next.js App Router, Recharts, Zod, better-sqlite3, DHT11 datasheet, MQ-135 datasheet, scikit-learn Decision Tree, statsmodels ARIMA, ArduinoJson.

Lampiran - Pertanyaan Umum & Jawaban Siap

- Q: Kenapa tidak memakai WebSocket?

A: Interval data 10 detik; polling 5 detik sudah memenuhi kriteria "kurang dari 5 detik". Polling stateless lebih sederhana, mudah di-debug, dan tidak perlu menjaga koneksi yang bisa putus.

- Q: Kenapa SQLite, bukan Supabase?

A: Data dari satu device, kebutuhan real-time sederhana, dan demo lokal. SQLite tanpa setup cloud, latensi nol, satu file. Untuk produksi multi-user tetap Supabase/Postgres.

- Q: Bagaimana threshold memengaruhi sistem?

A: Threshold tersimpan di tabel settings (default gas 500, temp 32 C, hum 75%), dipakai computeStatus di server, disinkronkan ke ESP32 tiap 30 detik, dan dipakai klasifikasi AI. Jadi satu nilai yang sama dipakai seluruh sistem.

- Q: Kalau WiFi mati, apa yang terjadi?

A: Logika lokal tetap jalan: ESP32 tetap mengevaluasi sensor dan menyalakan buzzer/LED/fan. Yang hilang hanya pencatatan ke server dan tampilan dashboard -- dashboard menandai OFFLINE setelah 30 detik tanpa data.

- Q: Bagaimana cara mengetahui klasifikasi AI akurat?

A: Klasifikasi berbasis aturan + training Decision Tree dari data riil berlabel berbasis threshold hasil EDA. Confidence dihitung dari skor. Ini bukan klaim akurasi mutlak, melainkan interpretable dan deterministik.

- Q: Kenapa prediksi pakai regresi linier?

A: Untuk tren jangka pendek (5 menit) regresi linier ringan, tanpa dependensi, dan mudah dijelaskan. Eksperimen jangka panjang memakai ARIMA di Python.

- Q: Apa langkah berikutnya?

A: Kontrol penuh multi-device, notifikasi Telegram saat alert, otentikasi pengguna, dan sensor tambahan (LDR/PIR) seperti di backlog PRD.